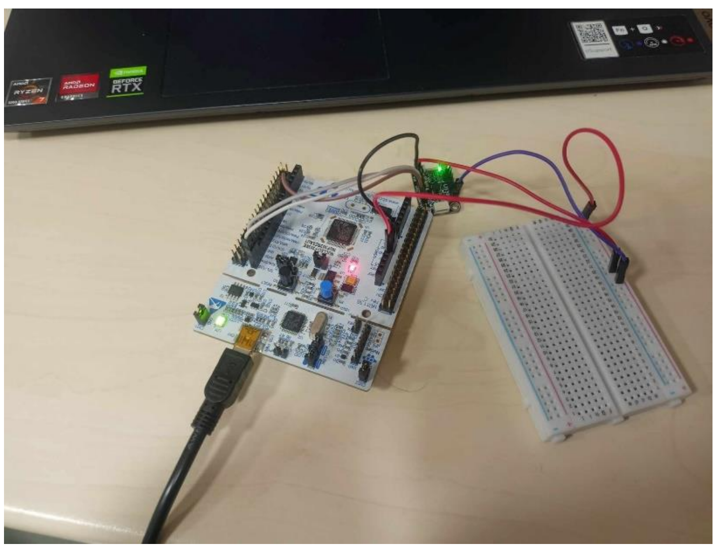
Real Time Monitoring System

Konrad Wołczański-Zub, Marek Kolończyk, Wojciech Pojasek, Bartosz Pudełko

Table of roles

Hardware	Konrad Wołczański-Zub, Wojciech Pojasek
STM configuration and C code	Konrad Wołczański-Zub, Bartosz Pudełko, Marek Kolończyk
Speech recognition neural network	Marek Kolończyk, Konrad Wołczański-Zub, Wojciech Pojasek
Main application	Konrad Wołczański-Zub, Bartosz Pudełko, Marek Kolończyk, Wojciech Pojasek

Real photo of the system with short description of how it works



STM32F303RE board is connected via COM to a laptop or PC. It reads analogue data from LIS3DH accelerometer converts it to digital and sends with a predefined sampling frequency via UART. MATLAB app receives the data and performs its real time analysis displaying it either in time waveforms or frequency spectrum. Type of the representation can either be changed by pressing the button in the app or using an inbuilt speech recognition neural network. Upon pressing record button and saying “time” or “spectrum” mode of the representation changes. For speech recognition, an inbuilt device microphone is used. Application also allows for changing the window length of STFT.

Description of elements of the source codes

C code

Main task of the C code is to initialize LIS3DH sensor and output its data via UART at a given frequency in a predefined format.

```c
void LIS3DH_Init(void)
{
  uint8_t data = 0x57;
  HAL_I2C_Mem_Write(&hi2c1, LIS3DH_ADDR, LIS3DH_CTRL_REG1,
                    I2C_MEMADD_SIZE_8BIT, &data, 1, HAL_MAX_DELAY);
}
```

```c
void LIS3DH_ReadXYZ(int16_t *x, int16_t *y, int16_t *z)
{
  uint8_t buffer[6];
  HAL_I2C_Mem_Read(&hi2c1, LIS3DH_ADDR, LIS3DH_OUT_X_L | 0x80,
                   I2C_MEMADD_SIZE_8BIT, buffer, 6, HAL_MAX_DELAY);

  *x = (int16_t)((buffer[1] << 8) | buffer[0]);
  *y = (int16_t)((buffer[3] << 8) | buffer[2]);
  *z = (int16_t)((buffer[5] << 8) | buffer[4]);
}
```

```c
while (1)
{
  int16_t x, y, z;
  char msg[64];

  LIS3DH_ReadXYZ(&x, &y, &z);
  sprintf(msg, "X:%d Y:%d Z:%d\r\n", x, y, z);
  printf("%s", msg);
  HAL_Delay(100);
}
```

In this example data is being sent via UART at a frequency of 10Hz in the format "X:... Y:... Z:...".

Speech recognition neural network

To create speech classifier words "time" and "spectrum" were recorded 10 times each. And MFCC data was extracted from them. Arrays for training the classifier were created and the classifier was trained.

```
for i = 1:samplesPerWord
    input(sprintf('Press ENTER to record sample %d of "%s"', i, word), 's');
    recObj = audiorecorder(fs, 16, 1);
    disp('Recording...');
    recordblocking(recObj, duration);
    disp('Done.');

    y = getaudiodata(recObj);
    filename = fullfile(folder, sprintf('%s_%d.wav', word, i));
    audiowrite(filename, y, fs);
    disp(['Saved to ', filename]);
end

features = [];
for i = 1:length(ads.Files)
    [x, fs] = audioread(ads.Files{i});
    coeffs = mfcc(x, fs, 'NumCoeffs', 13, 'LogEnergy', 'Ignore');
    features = [features; mean(coeffs)];
end

labels = ads.Labels;
classifier = fitcecoc(features, labels);
```

Classifier was saved to a separate file.

Matlab application

First public properties such as figures, labels and buttons were defined.

```
properties (Access = public)
    UIFigure                 matlab.ui.Figure
    FFTWindowEditField       matlab.ui.control.NumericEditField
    FFTWindowlengthsLabel    matlab.ui.control.Label
    ModeButtonGroup          matlab.ui.container.ButtonGroup
    SpectrumDomainButton     matlab.ui.control.RadioButton
    TimeDomainButton         matlab.ui.control.RadioButton
    COMPortEditField         matlab.ui.control.EditField
    COMPortLabel             matlab.ui.control.Label
    FsEditField              matlab.ui.control.NumericEditField
    FsHzEditFieldLabel       matlab.ui.control.Label
    StatusLabel              matlab.ui.control.Label
    RecordButton             matlab.ui.control.Button
    UIAxes                   matlab.ui.control.UIAxes
end
```

And then some private arrays and variables.

```
properties (Access = private)
    SerialObj
    Classifier
    Mode = "time";
    X = [];
    Y = [];
    Z = [];
    Timestamps = [];
    I = 0;
    PlotLines
    TimerObj
    Scale = 16064;
    fftWindow = 10;
    Fs = 10;
end
```

In the startup function speech classifier is loaded, timer and serial setups are done and time waveforms are being displayed.

```
function startupFcn(app)
    try
        load('trainedClassifier.mat', 'classifier');
        app.Classifier = classifier;
    catch
        app.StatusLabel.Text = 'Could not load classifier';
        app.Classifier = [];
    end
    app.setupSerial();
    hold(app.UIAxes, 'on');
    app.PlotLines(1) = plot(app.UIAxes, NaN, NaN, 'r');
    app.PlotLines(2) = plot(app.UIAxes, NaN, NaN, 'g');
    app.PlotLines(3) = plot(app.UIAxes, NaN, NaN, 'b');
    legend(app.UIAxes, {'X', 'Y', 'Z'});
    app.setupTimer();
end
```

Then few different events and functions are defined such as RecordButtonPushed, ModeButtonGroupSelectionChanged, COMPortEditFieldValueChanged, FsOrWindowValueChanged and perhaps the most interesting one recordAndClassify which is called upon RecordButtonPushed. It records 2s microphone input converts it to MFCC and classifies with previously created classifier on its basis figure remains in time domain or is switched to Fourier domain.

```
function recordAndClassify(app)
    app.StatusLabel.Text = "Recording...";
    rec = audiorecorder(16000, 16, 1);
    recordblocking(rec, 2);
    y = getaudiodata(rec);
    feat = mean(mfcc(y, 16000, 'NumCoeffs', 13, 'LogEnergy', 'Ignore'));
    label = predict(app.Classifier, feat);
    app.Mode = string(label);
    app.StatusLabel.Text = "Recognized: " + label;
end
```

Resulting exemplary figures

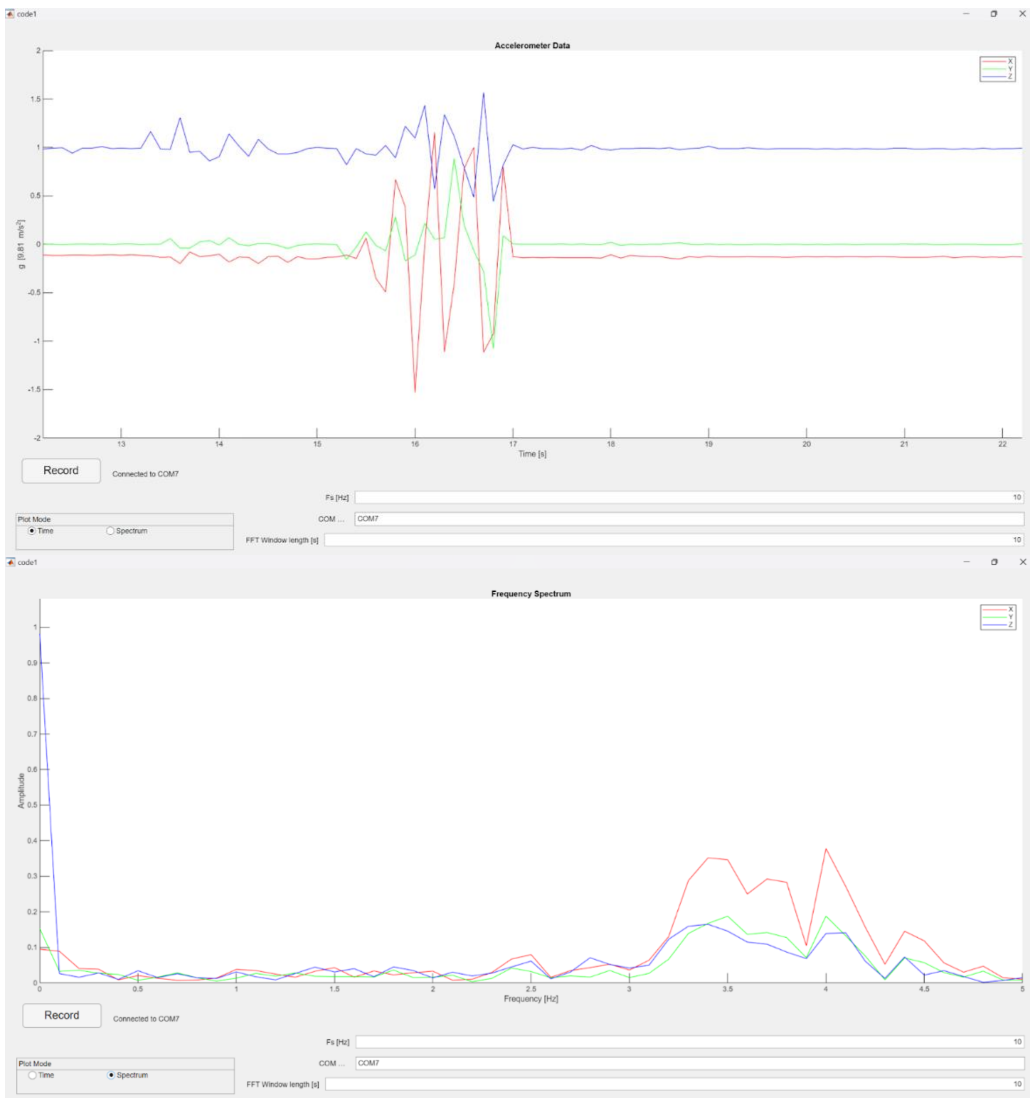


Figure 2. Frequency Spectrum

Interrupts

No interrupts were implemented.